

How Portfolio Workbench works

Live at <https://www.avisheksportfolio.com> · Deep analysis generated 2026-10-06 · app booted via npm run dev; screenshots are real captures · model claude-sonnet-5

Overview

Portfolio Workbench is a personal project-tracking site for Avishek Chandra: a Git-backed system where each project is a folder containing an `index.md` (YAML front matter plus prose) and an `artifacts/` directory of images, PDFs, and videos. Astro's content collections load and validate these records at build time with a Zod schema (`project-schema.ts`), then render a static homepage carousel and one detail page per project. There is no database — the Markdown file is the single source of truth, and Git history is the audit trail.

The same codebase runs in two modes. In local development (`npm run dev`), a full editor is available at `localhost:4321`: creating or editing projects writes directly to the filesystem through a Vite middleware bound to the loopback interface, and editing private projects requires a WebAuthn device unlock (Windows Hello, fingerprint, or PIN) implemented in `device-auth.ts`. In the hosted build (`PUBLIC_ONLY=1`, deployed to GitHub Pages as an installable PWA), there is no server at all — every edit is a commit made from the browser through the GitHub contents API, using a fine-grained personal access token entered once via a "Connections" dialog in the header.

The site is meant for one user (Avishek) to maintain a running, honest log of what he's built and why, and for visitors to browse that log. Each project record carries a status (idea/dev/review/delivered — a project can't be more than "idea" without a linked repository), a privacy level (public/private), dates, and optional repository/PR/site links. Private projects live in a separate `workbench-private` GitHub repo; the public build generates "locked stub" cards for them (real id/dates, generic text) and the real content is fetched client-side only after the visitor authenticates with GitHub, via the `PrivateVault` component.

A distinguishing feature is the AI-assisted documentation pipeline. Clicking "Draft from repository" gathers evidence from the linked GitHub repo/PR (README, diffs, commits, issues — owner-allowlisted and secret-redacted), asks an LLM (Anthropic directly, or OpenAI through a bundled Cloudflare Worker proxy) to rewrite the description and why, and then commits an `analysis-requests/<folder>.json` file that triggers a GitHub Actions workflow (`deep-analysis.yml`). That workflow clones the linked repo, boots it (via docker compose, Dockerfile, npm, or python heuristics), crawls it with Playwright, screenshots every page, asks the model to document every control, typesets a PDF, and commits everything back into the project's artifacts — which is, recursively, how documents like this one are produced for Portfolio Workbench's own sibling projects.

Architecture

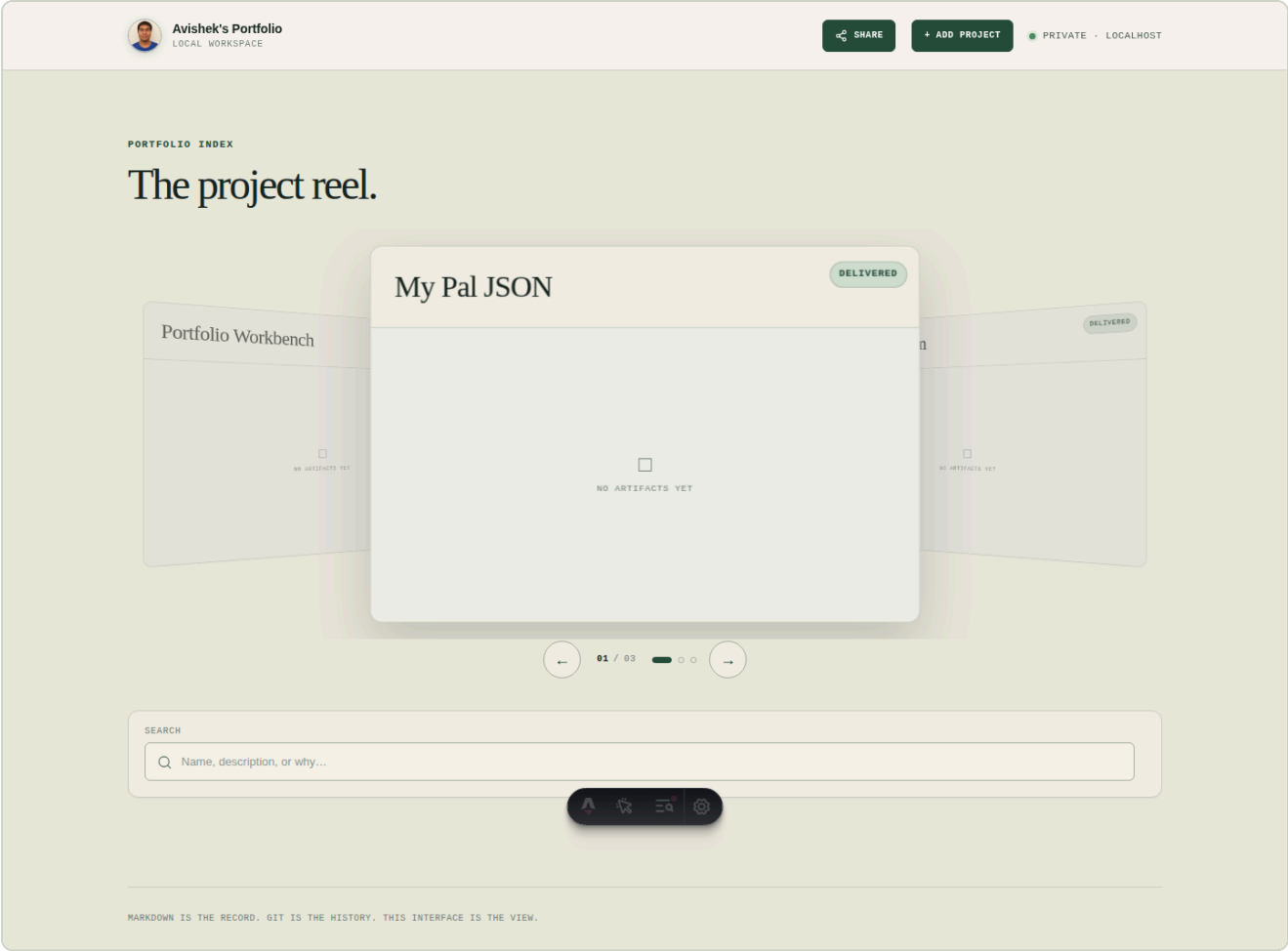
The stack is Astro 7 + TypeScript, output as a static site, with `astro:content` collections backed by a Zod schema (`project-schema.ts`) validating every project's front matter at build time. Styling is a single global CSS file; there is no client framework — interactivity is implemented with vanilla TypeScript modules under `src/lib/` that are loaded as inline `<script>` blocks inside Astro components (`ProjectCard`, `RemoteEditor`, `RemoteCreate`, `ProjectDelete`, `PrivateVault`). `BaseLayout.astro` provides the shared header (brand/contact-card button, Share button, "+ Add project" link, GitHub connections icon) and footer across all pages.

Local-mode editing is implemented as a Vite dev-server plugin (`local-editor.ts`) that intercepts requests on the loopback interface, validates payloads with `project-draft/update` Zod schemas, and writes directly to `src/content/projects/<id>-<slug>/index.md` and its `artifacts/` folder; private-project writes additionally require a WebAuthn ceremony handled by `device-auth.ts` (registration/authentication via `@simplewebauthn/server`, with an unlock cookie). Hosted-mode editing (`github-client.ts`) talks directly to the GitHub REST Contents API from the browser using a PAT stored in `localStorage`; `RemoteEditor` patches fields in the public repo (`project-workbench`), while `PrivateVault` patches the same shape of data in the private repo (`workbench-private`), sharing editor logic via `project-editor.ts` and `field-editor.ts` (themed modal dialogs per field, with optional inline AI-draft/undo).

Artifacts (any file type) are rendered through a single shared carousel module (`artifact-carousel.ts`) used both at build time (public pages, server-rendered) and at runtime (private vault, injected after unlock) so the two experiences look identical; images/videos play inline, PDFs get pdf.js first-page previews, and PowerPoints are auto-converted to PDF by a GitHub Action (`convert-pptx.yml`). The AI subsystem is layered: `ai/evidence.ts` collects and redacts a budgeted evidence package from the GitHub API, `ai/provider.ts` abstracts Anthropic vs. OpenAI (OpenAI routed through `proxy/openai-worker.js` on Cloudflare Workers because it blocks browser CORS), `ai-complete.ts` adds an owner-allowlist safety check and the field-specific prompts, and `ai/repo-draft.ts` plus `scripts/deep-analysis.mjs` orchestrate the two-stage draft-then-deep-analysis flow, the latter run entirely in CI (`deep-analysis.yml`) using Playwright and the Anthropic SDK, polled by `ai/analysis-status.ts` on the client. Deploys are fully automated: every push to `main` runs `generate-private-stubs.mjs` (using a `PRIVATE\_REPO\_TOKEN` secret) then builds with `PUBLIC\_ONLY=1` and publishes to GitHub Pages under the custom domain `avisheksportfolio.com` (`deploy.yml`).

Page: /

The portfolio's home page — a browsable, searchable reel of all public (and, in local/dev mode, private) projects, each represented by a card in a coverflow-style carousel.



Controls & fields

CONTROL / FIELD	WHAT IT DOES
Contact card for Avishek Chandra (button)	Opens a business-card-style panel/dialog with Avishek's name, email, phone, and certification links (defined as the `CARD` object in `BaseLayout.astro`); toggled via `data-card-open`.
Share (button)	Opens a share dialog (native share sheet or copy-link fallback) for the current page, wired via `data-share-open` in `BaseLayout.astro`.
+ Add project (a)	Navigates to `/manage`, the project-creation page (`withBase('/manage')` link in the header).
My Pal JSON / Deodapper.com /	Each project name links to its detail page at `/projects/<slug>` (e.g., `/projects/my-pal-json`); clicking the card's artifact area links to `/projects/<slug>#edit`. Rendered by `ProjectCard.astro`.

Portfolio Workbench (a, project card links)	
☐ No artifacts yet (a)	For a project with no uploaded artifacts, links to that project's detail page (<code>`ProjectCard.astro`'s empty-state branch</code>).
Show previous project (button, `data-project-coverflow-previous`)	Steps the carousel to the previous visible project card via <code>`showCoverflowCard(currentIndex - 1)`</code> in the inline script.
Show My Pal JSON / Show Deodapper.com / Show Portfolio Workbench (button, `data-project-coverflow-dot`)	Each dot jumps the carousel directly to that project's card (<code>`showCoverflowCard(card)`</code>), updating the aria-current state and position counter.
Show next project (button, `data-project-coverflow-next`)	Steps the carousel to the next visible project card.
Name, description, or why... (input[search], `data-filter-search`)	Free-text filter; on input, matches against each card's precomputed <code>`data-search`</code> string (name+description+why lowercased) to show/hide cards, update the empty-results panel, and refresh the coverflow/dots to only the visible subset.
Clear filters (button, `data-reset-filters`, hidden until a filter is active)	Resets the search field and re-shows all project cards.
Clear all filters (button,	Same reset behavior, triggered from the "No matching projects" state.

``data-empty-
reset`,
shown only
in the
empty-
results
panel)`

**Unlock with
Windows
Hello
(button,
`data-
private-
unlock-
button`)**

Invokes the device-unlock client (``device-unlock-client.ts``) to perform a WebAuthn authentication ceremony against ``device-auth.ts``, then navigates to the pending private project URL on success.

Page: /manage

Form for creating a new project record — either committed to GitHub (hosted site) or written to the local filesystem (local dev workbench), depending on build mode.

**Avishek's Portfolio**
LOCAL WORKSPACE

[SHARE](#) [ADD PROJECT](#) PRIVATE · LOCALHOST

[PORTFOLIO INDEX](#)

LOCAL EDITOR

Add a project.

Capture what it is, why you built it, and where it stands.

- Writes to your project folder**
Available while `npm run dev` is running. Creating asks for your device unlock.

01

Project

The small set of details worth keeping.

NAME

DESCRIPTION

WHY IT WAS BUILT



02

State

Just enough context to understand where the project stands.

STATUS

Choose status

PRIVACY

Choose privacy

START DATE

10/06/2026

Create project

MARKDOWN IS THE RECORD. GIT IS THE HISTORY. THIS INTERFACE IS THE VIEW.

Controls & fields

CONTROL / FIELD	WHAT IT DOES
--------------------	--------------

+ Add project (a, header)	Same global header link to <code>`/manage`</code> (already on this page; acts as a no-op/self-link).
Repository link (input[url], , `name="repositoryUrl", hosted mode only)	GitHub repo URL; enables the "Draft from repository" button when it matches a <code>`github.com/owner/repo`</code> pattern, and is stored on the new project record. Without it, the project's status is constrained to "idea" per the schema/template rule.
+ Draft from repository (button, `data-create-ai`, hosted mode)	Calls <code>`draftFromRepository()`</code> to collect evidence from the repo/PR and auto-fill Name/Description/Why via the AI provider; disabled until a valid repo URL is entered.
Pull request link (input[url], , `name="pullRequestUrl", optional, hosted mode)	Optional PR URL, also used as AI drafting evidence and stored on the record.
Name (input, `name="name", required, maxLength 120)	The project's display name; used to generate the slug (<code>`slugify`</code>) and as the <code>`name`</code> front-matter field. Validated against the <code>`projectDraftSchema`/`projectUpdateSchema`</code> (min 1 character).
A useful name for the project (local-mode input, `name="name")	Same field in the local-editor form variant; validated by <code>`local-editor.ts`</code> 's Zod-based <code>`createProject`</code> flow.
Description / What	One-to-few-sentence description of the project; required, max 500 chars.

does it do? (textarea, `name="description", required, maxlength=500)	
Why it was built / What problem, curiosity, or need made you build it? (textarea, `name="why", required, maxlength=3000)	First-person rationale text; required, max 3000 chars.
Status / Choose status (select, `name="status", required)	One of `idea dev review delivered` (`projectStatuses`); required, validated by schema.
Privacy / Choose privacy (select, `name="privacy", required, local mode only)	One of `private public` (`privacyLevels`); hosted-site creations are always public (no privacy selector shown in `RemoteCreate`), set server-side for local mode.
Start date (input[date], `name="startDate", required, defaults to today)	ISO date string matching `YYYY-MM-DD`, stored as the project's `startDate`.
Create project →	Submits the form; hosted mode commits a new `src/content/projects/<slug>/index.md` file via `putFile()` in `github-client.ts` after a duplicate-repo check; local mode performs a device-unlock ceremony then POSTs to the local-editor middleware (`local-editor.ts` → `createProject()`) which

(button[submit])	validates with `projectDraftSchema`, slugifies the name, checks for slug collisions, and writes the new folder/`index.md`.
-------------------------	--

Restore it instead / Open the existing project / Add as a new project anyway / Cancel (dialog buttons, `data-duplicate-*`, hosted mode)	Shown when the entered repository URL already belongs to an existing (possibly deleted) project; lets the user restore the soft-deleted original, navigate to the existing project page, force-create a duplicate anyway, or cancel the creation.
--	---

Page: /projects/my-pal-json

Detail/edit page for the "My Pal JSON" project — displays its description, why, status, dates, links, and artifacts, and (for the owner, once connected) allows inline editing of every field plus media management.

Avishek's Portfolio

LOCAL WORKSPACE

SHARE

ADD PROJECT

PRIVATELOCALHOST

PORTFOLIO INDEX

PROJECT / 004

DELIVERED

My Pal JSON

My Pal JSON is a browser-based API client that stores request collections in Git repositories, lets you send API calls and analyze JSON responses, and generates model classes in six programming languages. It runs locally so your localhost APIs and secrets never leave your machine, and supports chaining requests together with variable passing between responses.

PRIVACY

Public

Ready to share.

STATUS	STARTED	UPDATED	REPOSITORY	PULL REQUEST	LIVE SITE
Delivered	Oct 3, 2024	Sep 27, 2026	github.com/aviman1258/my-pal-json	—	—

LOCAL EDITOR

Manage this project

Changes are saved directly to this project's Markdown record.

✚ Edit project details

Why it was built

I built this project to eliminate the friction of context-switching between multiple tools during API development. As someone who constantly moved between a request client, JSON formatter, structure inspector, and code generator, I realized each context switch broke my focus and slowed down exploration—so I set out to consolidate all these workflows into a single lightweight browser application. What started as a productivity fix evolved into something more interesting: a workspace that runs entirely locally, integrates with git repositories to sync API collections with teammates using Postman, and chains requests together visually to simulate complex API flows. I wanted to prove that the right tool design could make API exploration feel seamless rather than fragmented.

Evidence from the work

Images, documents, and recordings stored alongside this project. Drop files onto the carousel to add them, drag the tiles to reorder.

✚ Add media

↓

No artifacts attached yet

Drop images, PDFs, or videos here — or click to browse your files.

MARKDOWN IS THE RECORD. GIT IS THE HISTORY. THIS INTERFACE IS THE VIEW.

Controls & fields

CONTROL / FIELD	WHAT IT DOES
Contact card for Avishek Chandra (button)	Same global header business-card dialog as on every page.
Share (button)	Same global header share dialog, sharing this project's detail page URL.
+ Add project (a)	Navigates to `/manage`.
github.com /aviman1258/my-pal-json (a)	External link to the linked GitHub repository (`data.repositoryUrl`), opened in a new tab.
✦ Edit project details (summary, details element)	Expands/collapses the inline project-editor panel wrapping the editable fields (local-mode styling reused; actual mechanics driven by `RemoteEditor`'s field-pencil wiring on the hosted site).
name (input)	Editable project name field inside the (local-editor-styled) edit panel; on hosted site, actual edits go through the pencil-triggered `openFieldEditor` modal (`project-editor.ts` `PROJECT_FIELDS.name`, maxLength 120) which patches `name` in front matter via `makeFrontmatterPatcher` and commits to GitHub.
Description textarea ("My Pal JSON is a browser-based API client...")	Editable description text (max 500 chars); saved via the same field-editor/patch mechanism (`PROJECT_FIELDS.description`).
Why textarea ("I built this project to eliminate the friction...")	Editable why-text (max 3000 chars), same mechanism (`PROJECT_FIELDS.why`), with an optional AI-draft button inside the modal that re-drafts from repo evidence.
Choose status select (idea/dev/review/delivered)	Edits the `status` field via the status-pill variant of the field editor (`PROJECT_FIELDS.status`); writes `status` into front matter.
Choose privacy	Edits the `privacy` level of the project (local-editor-styled field; on hosted mode this is generally fixed/derived from which repo the record lives in).

select (private/public)	
startDate (input[date])	Edits the project's start date (<code>PROJECT_FIELDS.startDate`</code> ), stamped alongside an auto-updated <code>`updatedAt`</code> on every save.
Save changes (button[submit])	Commits whichever field(s) were modified in the (local-style) edit form; on the hosted site the actual commit path is per-field via pencils/modals, applied through <code>`backend.putIndex()`</code> (GitHub contents PUT) in <code>`project-editor.ts`</code> .
+ Add media (button)	Opens the hidden file input (<code>`data-remote-media-input`</code>) to pick one or more artifact files (any extension, ≤ 25 MB each); on change, each file is uploaded via <code>`putFile()`</code> to <code>`src/content/projects/<folder>/artifacts/<filename>`</code> on GitHub.
↓ No artifacts attached yet / drop zone (button)	Acts as both a dropzone (drag-and-drop images/PDFs/videos) and a click-to-browse trigger for the same hidden file input, per <code>`carouselHtml()`</code> 's empty-state and <code>`data-media-dropzone`</code> wiring.
Star (artifact-slide feature button, <code>`data-media-feature-trigger`</code>, present once artifacts exist)	Marks that artifact as the project's <code>`featuredArtifact`</code> in front matter, determining which image shows as the card hero on <code>`/`</code> .
Trash (artifact-slide delete button, <code>`data-media-delete-trigger`</code>, present once artifacts exist)	Deletes that artifact file from the repo via <code>`deleteFile()`</code> after a confirmation dialog.
Draft from repo (pill button, <code>`data-repo-</code>	Opens a confirmation dialog explaining the two-stage AI draft + background deep-analysis flow; confirming calls <code>`draftFromRepository()`</code> to rewrite description/why and commits an <code>`analysis-requests/<folder>.json`</code> file to queue the CI job.

**draft`,
shown
once
GitHub is
connected
and a repo
link exists)**

**Delete
project
(fixed
trash pill,
`ProjectDelete.astro`,
visible
once
connected)**

Opens a confirm dialog; confirming sets `deleted: true` in the project's front matter (soft delete) via `putFile()`, then redirects home after ~1.8s.

Page: /projects/tour-guide

Detail/edit page for "Deodapper.com" (Tour Guide) — same structure and editing machinery as other public project pages, showing its description, links (including a live site URL), and artifacts.

Avishek's Portfolio

LOCAL WORKSPACE

SHARE

ADD PROJECT

PRIVATELOCALHOST

PORTFOLIO INDEX

PROJECT / 005

DELIVERED

Deodapper.com

www.deodapper.com Tour Guide creates self-drive day trip itineraries from an airport or starting point to a hotel, optimized around your interests and arrival time. The installable web app uses GPS to follow you in the car, narrating each planned stop and interesting landmarks you pass through text-to-speech. It uses Claude to propose attractions grounded in Wikipedia and map data, routes the day with real timings, and runs offline to guide you with narration.

PRIVACY

Public

Ready to share.

STATUS	STARTED	UPDATED	REPOSITORY	PULL REQUEST	LIVE SITE
Delivered	Sep 9, 2026	Sep 27, 2026	github.com/aviman1258/tour-guide	—	deodapper.com

LOCAL EDITOR

Manage this project

Changes are saved directly to this project's Markdown record.

Edit project details

Why it was built

I built Deodapper because the friction of arriving in a new city shouldn't mean you miss exploring it. When you land at an airport or start a drive from anywhere, you want to discover something interesting on the way to your hotel, but the planning is scattered—finding stops that match your interests, timing them to your check-in, navigating while spotting things you pass—all across different apps and tabs. I wanted to explore how AI planning, real-time routing, and mobile-first design could turn a travel day into something less logistical and more like having a local friend in the passenger seat, so I stress-tested it on the road to Houston in November 2026.

Evidence from the work

Images, documents, and recordings stored alongside this project. Drop files onto the carousel to add them, drag the tiles to reorder.

Add media

No artifacts attached yet

Drop images, PDFs, or videos here — or click to browse your files.

MARKDOWN IS THE RECORD. GIT IS THE HISTORY. THIS INTERFACE IS THE VIEW.

Controls & fields

CONTROL / FIELD	WHAT IT DOES
Contact card for Avishek Chandra (button)	Global header business-card dialog, same as all pages.
Share (button)	Global header share dialog for this project's URL.
+ Add project (a)	Navigates to `/manage`.
www.deodapper.com (a, in description text)	Linkified auto-generated hyperlink from the description text (via `linkify()`) pointing to the live app.
github.com/aviman1258/tour-guide (a)	External link to the linked GitHub repository.
deodapper.com (a, metadata strip "Live site" row)	External link to `data.siteUrl`, the deployed Tour Guide/Deodapper app.
+ Edit project details (summary)	Expands the inline project-editor panel, same as on other project pages.
name (input)	Editable project name, same `PROJECT_FIELDS.name` mechanism as My Pal JSON.
Description textarea ("www.deodapper.com Tour Guide creates self-drive day trip itineraries...")	Editable description (max 500 chars) via field editor.
Why textarea ("I built Deodapper because the friction of arriving in a new city...")	Editable why-text (max 3000 chars) via field editor, with AI-draft support.
Choose status select	Edits project status (idea/dev/review/delivered).
Choose privacy select	Edits privacy level (private/public).
startDate (input[date])	Edits the start date field.
Save changes (button[submit])	Same commit mechanism as other project pages — per-field patch via GitHub contents PUT.
+ Add media (button)	Opens file picker to upload new artifacts for this project.
↓ No artifacts attached yet / drop zone (button)	Dropzone/click-to-browse for uploading images, PDFs, or videos.

Star / Trash on artifact slides (once present)	Feature or delete an uploaded artifact, same as other project pages.
Draft from repo (pill)	Re-drafts description/why from the linked repo and queues the background deep-analysis CI run for this project.
Delete project (fixed trash pill)	Soft-deletes this project record via confirmation dialog.

Page: /projects/porfolio-workbench

Detail/edit page for "Portfolio Workbench" itself — describing the app being documented here, with the same public-project editing machinery as its siblings.

Avishek's Portfolio

LOCAL WORKSPACE

SHARE

ADD PROJECT

PRIVATE · LOCALHOST

PORTFOLIO INDEX

PROJECT / 003

DELIVERED

Portfolio Workbench

Portfolio Workbench is a Git-backed portfolio site built with Astro where each project is a Markdown file validated and rendered at build time. It provides two editing interfaces: a local workbench with device authentication for offline editing, and a remote editor that commits changes to GitHub for the live site. All project data, artifacts, and metadata stay in version-controlled files without any database.

PRIVACY

Public

Ready to share.

STATUS	STARTED	UPDATED	REPOSITORY	PULL REQUEST	LIVE SITE
Delivered	Sep 1, 2026	Sep 7, 2026	github.com/aviman1258/project-workbench	—	avisheksportfolio.com

LOCAL EDITOR

Manage this project

Changes are saved directly to this project's Markdown record.

Edit project details

Why it was built

I built this to keep my project records in Git—as plain Markdown and YAML files—rather than locked into a hosted service or database. As a senior software engineer shipping side projects, I wanted a durable, diffable archive I could version-control, inspect by hand, and redeploy without migration pain. The local-only editor and passkey-based privacy controls let me document experiments and ideas end-to-end while staying selective about what's shareable, and the static build output means the portfolio itself stays simple and portable.

Evidence from the work

Images, documents, and recordings stored alongside this project. Drop files onto the carousel to add them, drag the tiles to reorder.

Add media

↓

No artifacts attached yet

Drop images, PDFs, or videos here — or click to browse your files.

MARKDOWN IS THE RECORD. GIT IS THE HISTORY. THIS INTERFACE IS THE VIEW.

Controls & fields

CONTROL / FIELD	WHAT IT DOES
Contact card for Avishek Chandra (button)	Global header business-card dialog.
Share (button)	Global header share dialog for this page's URL.
+ Add project (a)	Navigates to `/manage`.
github.com/aviman1258/project-workbench (a)	External link to this project's own GitHub repository.
avishekspportfolio.com (a, metadata strip "Live site" row)	External link to `data.siteUrl`, the live production site (self-referential — this is the app being documented).
✦ Edit project details (summary)	Expands the inline editor panel.
name (input)	Editable project name field (`PROJECT_FIELDS.name`).
Description textarea ("Portfolio Workbench is a Git-backed portfolio site built with Astro...")	Editable description (max 500 chars).
Why textarea ("I built this to keep my project records in Git—as plain Markdown and YAML files...")	Editable why-text (max 3000 chars), with AI-draft support.
Choose status select	Edits status (idea/dev/review/delivered).
Choose privacy select	Edits privacy (private/public).
startDate (input[date])	Edits the project's start date.
Save changes (button[submit])	Commits field edits via GitHub Contents API PUT, same mechanism as other project pages.
+ Add media (button)	Opens the file picker to attach new artifacts (e.g., this very analysis report) to the project.

↓ No artifacts attached yet / drop zone (button)	Dropzone/click-to-browse alternative for uploading artifacts.
Star / Trash on artifact slides (once present)	Feature or delete an uploaded artifact for this project.
Draft from repo (pill)	Re-runs AI drafting of description/why from this very repository and queues a background deep-analysis CI run (the mechanism that generated reports like this one).
Delete project (fixed trash pill)	Soft-deletes this project record (marks `deleted: true`), removing it from the live site after the next rebuild while preserving its Git history.

Generated by the workbench deep-analysis pipeline from the repository source and a live crawl.